GAME CLIENT PORTFOLIO · TEAM PROJECT

CollectingRPG

모바일 수집형 RPG (오토배틀 · 로그라이크 맵) — 브랜치 통합 관리 · UI 연출 프로토타입

Unity 2022.3 LTS | C# | URP | DOTween | Addressables | UniTask | Firebase | Git 브랜치 전략

김동준 · dkdl01028@icloud.com · github.com/kimdongjun22

기간	2025.08.14 ~ 2025.10.16 (약 2개월)	**인원**	6~7인 팀 프로젝트
담당	브랜치 통합(merge) 관리, DOTween UI 연출 프로토타입	**저장소**	github.com/HSD06040/CollectingRPG

6~7인
팀 · 약 2개월

5회
develop 브랜치 통합

7개
UI 연출 스크립트 (KDJ_UI)

416개
프로젝트 전체 C# 규모

프로젝트 소개

가챠로 캐릭터를 모으고, 강화·편성한 뒤 **배치와 시너지로 싸우는 자동 전투**와 **노드형 로그라이크 맵**을 진행하는 모바일 수집형 RPG입니다. 6~7명이 약 2개월 동안 개발한 중규모 프로젝트로, C# 파일 416개 규모입니다.

구분	내용
수집·육성	가챠, 캐릭터·마법석 강화, 파티 편성
전투	그리드 배치, 클래스·유닛 시너지, 증강, 유닛 FSM, 스킬·버프·투사체
맵	노드형 맵 생성, 맵 이벤트, 스테이지 상점
아웃게임	Firebase 로그인(구글/게스트), 퀘스트, 우편함, 일일상점, 인앱결제, 광고
기술 기반	Addressables 원격 리소스, UniTask 비동기, 자동 초기화 싱글톤 매니저, 오브젝트 풀

담당 역할

① 브랜치 통합 관리

팀원들이 `feat/*` 기능 브랜치에서 작업한 결과를 `develop`으로 합치는 **통합 작업**을 맡았습니다. 프로젝트 초기(08.18 ~ 08.21)에 아래 5건을 병합했고, 모두 충돌 없이 `develop`에 반영되었습니다.

날짜	통합 내용
08.18	`feat/CharComposition` (캐릭터 편성) → `develop`
08.18	`main` → `feat/Sign-in` (로그인 브랜치 최신화)
08.21	`JJM` (UI) → `feat/Unit`
08.21	`feat/CharComposition` → `develop`

날짜	통합 내용
08.21	`feat/Unit` (유닛) → `develop`

이 과정에서 기능 브랜치 → 통합 브랜치(`develop`) → 배포 브랜치(`main`)로 이어지는 **Git 브랜치 전략**과, 병합 전에 브랜치를 최신 상태로 맞추는 흐름을 익혔습니다.

② DOTween UI 연출 프로토타입 (`KDJ_UI` 브랜치)

게임 UI에 쓸 **팝업·버튼 연출**을 DOTween으로 실험한 프로토타입입니다. 개인 브랜치에서 작업했고 `develop`에는 병합되지 않았습니다.

- `PanelHandler` : DOTween `Sequence`로 팝업이 커졌다 줄어들며 열리고 닫히게 함. 닫힘 애니메이션이 끝나면 `OnComplete` 콜백으로 비활성화
- `ButtonHandler` / `CloseButtonHandler` : 버튼을 누르면 눌림 효과(0.95 → 1.05 → 1.0)를 준 뒤 팝업 열기/닫기
- `MainMenu` : 메인 메뉴 버튼 핸들러(종료 처리는 `#if UNITY_EDITOR`로 에디터와 빌드를 나눔)
- `Projectile` : `DOJump`로 포물선 궤적 발사체 테스트

대표 코드

① DOTween Sequence 팝업 열기/닫기 — PanelHandler.cs

```
public void Show()
{
    gameObject.SetActive(true);
    var seq = DOTween.Sequence();
    seq.Append(transform.DOScale(1.1f, 0.2f));   // 살짝 크게
    seq.Append(transform.DOScale(1f, 0.1f));     // 원래 크기로 안착
    seq.Play();
}

public void Hide()
{
    var seq = DOTween.Sequence();
    transform.localScale = Vector3.one * 0.2f;
    seq.Append(transform.DOScale(1.1f, 0.1f));
    seq.Append(transform.DOScale(0.2f, 0.2f));
    seq.Play().OnComplete(() => { gameObject.SetActive(false); });  // 연출이 끝난 뒤 비활성화
}
```

② 버튼 눌림 효과 후 팝업 호출 — ButtonHandler.cs

```
public void OnButtonClick()
{
    var seq = DOTween.Sequence();
    seq.Append(transform.DOScale(0.95f, 0.1f));
    seq.Append(transform.DOScale(1.05f, 0.1f));
    seq.Append(transform.DOScale(1f, 0.1f));
    seq.Play().OnComplete(() => { popupWindow.Show(); });  // 눌림 연출 → 팝업 열기
}
```

회고

- **배운 점**: 6~7명 규모 프로젝트에서 기능 브랜치를 통합하며 Git 브랜치 전략이 실제로 어떻게 돌아가는지 경험했습니다. DOTween `Sequence`와 `OnComplete` 콜백으로 **연출이 끝나는 시점에 상태를 바꾸는 방법**을 익혔습니다.
- **아쉬운 점**: UI 연출 작업을 개인 브랜치에서 끝까지 다듬어 PR로 올리지 못해, 최종 게임에 반영되지 않았습니다. 작업 브랜치에 다른 SDK 파일이 함께 딸려 들어가는 등 커밋 범위 관리도 부족했습니다.
- **다음에는**: 작은 단위로 자주 PR을 올리고, 컴파일·동작을 확인한 뒤 리뷰를 요청하겠습니다. 프로젝트 공용 팝업 매니저(`PopupManager`)에 연출을 붙이는 방향으로 통합을 먼저 설계하겠습니다.